

OmniFlow — Phase 3 (P2P Orchestrator) Audit Charter

Purpose: the constraints the Principal Council will grade against. Scaffold to these so Phase 3 passes on first review.

What the service is

Consumes `omniflow.orchestration.v1` (classified events from CommBot), runs each purchase order through a **multi-tier approval DAG**, persists workflow state durably, and emits approved/rejected outcomes onward — surviving pod crashes mid-approval.

Non-negotiable design constraints (graded)

1. DAG correctness

- Model approvals as a directed graph; **detect and reject cycles** at construction (a cyclic approval chain is a terminal, DLQ-worthy config error — not a hang).
- **Topological sort via Kahn's algorithm, $O(V+E)$** ; deterministic tie-breaking. Persist the computed order / ready-set — do not recompute the full sort on every event.
- Pure DAG engine lives in `core/domain`, free of Kafka/CRDB imports.

2. Durable checkpointer (resume-on-crash)

- Per-workflow state persisted in CockroachDB. After **every** node transition, checkpoint **atomically** (state update + any emitted event in one tx — no dual-write; reuse the outbox+changefeed pattern).
- On pod restart, resume from the last committed checkpoint. Node execution is **at-least-once**, therefore every node action **MUST** be **idempotent**, guarded by an execution ledger keyed on `(workflow_id, node_id, attempt)`.

3. Multi-pod safety

- Multiple orchestrator replicas will run. Prevent two pods advancing the same workflow: **workflow-level lease/lock** (`SELECT ... FOR UPDATE` or a TTL lease row). No optimistic advance without it.

4. Human-in-the-loop

- A node may **suspend** awaiting human approval and **resume** on an external event/timer. State must be durable across an arbitrary wait; no in-memory-only waits.

5. Telemetry & downstream continuity

- One span per node; `workflow_id` + `aggregate_id` as trace attributes; propagate the inbound W3C traceparent.
- Carry `sequence_engine_key` / HLC emit-time forward so the WebGL 3D timeline stays ordered and Power BI latency math stays trivial — consistent with Phase 1/2.

Expected package layout (hexagonal)

services/p2p-orchestrator/

cmd/main.go

internal/core/domain/

dag.go # graph, Kahn topo sort, cycle detection

workflow.go # approval state machine + transitions

internal/core/ports/

state_store.go # LoadWorkflow / Checkpoint(tx) / AcquireLease

publisher.go # transactional outbox emit

internal/adapters/inbound/kafka/consumer.go # consumes omniflow.orchestration.v1

internal/adapters/outbound/crdb/ # checkpointer + lease + outbox

infrastructure/storage/orchestrator_schema.sql # workflow_state, node_execution ledger,
outbox + changefeed

Predictable failure modes I will hunt (pre-warned)

- Topo sort that silently drops nodes on a cycle instead of erroring.
- Checkpoint written in a separate tx from the emitted event (dual-write reintroduced).
- "Resume" that replays already-executed nodes because there's no execution ledger.
- In-memory workflow map that evaporates on pod restart (no real persistence).
- No lease → two pods double-approve the same PO.
- Recomputing the full topological sort on every single event ($O(V+E)$ per event at scale).